
Markov Decision Process - II

CS5491: Artificial Intelligence
ZHICHAO LU

Content Credits: [Prof. Wei](#)'s CS4486 Course
and [Prof. Boddeti](#)'s AI Course



TODAY

Value Iteration

Policy Iteration

- Policy Evaluation
- Policy Extraction



EXAMPLE: GRID WORLD

A maze like problem

- The agent lives in a grid
- Walls block the agent's path

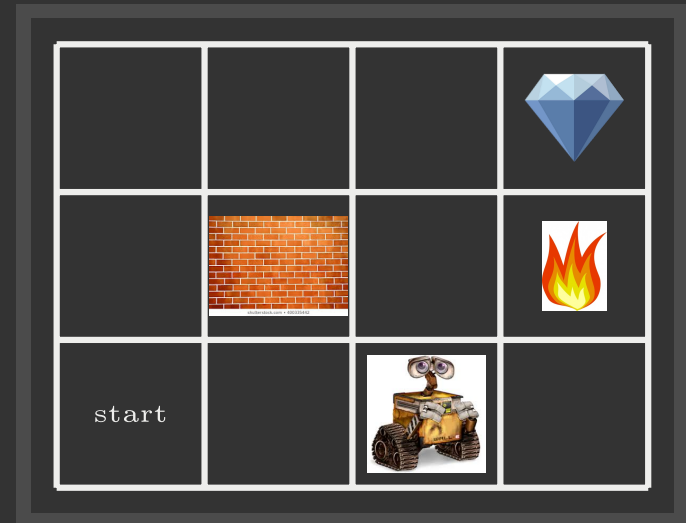
Noisy movement: actions do not always go as planned

- 80% of the time, the action North takes the agent North (if there is no wall there)
- 10% of the time, North takes the agent West; 10% East
- If there is a wall in the direction the agent would have been taken, the agent stays put

The agent receives rewards each time step

- Small "living" reward each step (can be negative)
- Big rewards come at the end (good or bad)

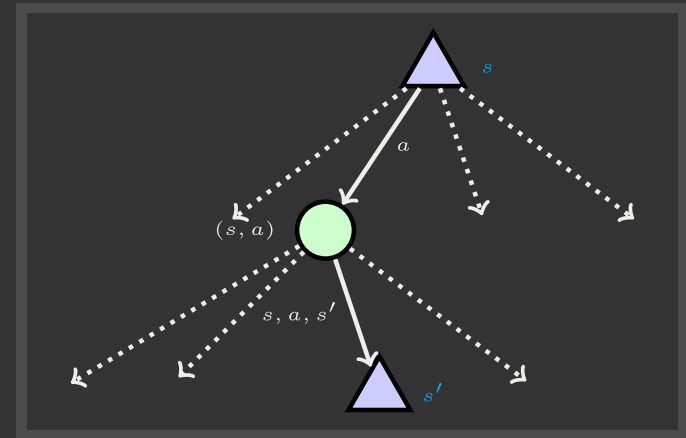
Goal: maximize sum of rewards



RECAP: DEFINING MDPS

Markov decision processes:

- Set of states $\mathcal{S}$
- Start state s_0
- Set of actions $\mathcal{A}$
- Transitions $P(s'|s, a)$ (or $T(s, a, s')$)
- Rewards $R(s, a, s')$ (and discount γ)



MDP quantities so far:

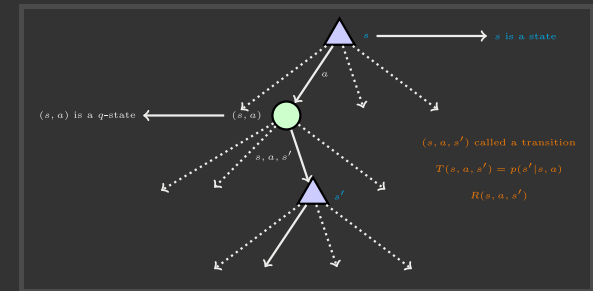
- Policy = Choice of action for each state
- Utility = sum of (discounted) rewards

SOLVING MDPS



OPTIMAL QUANTITIES

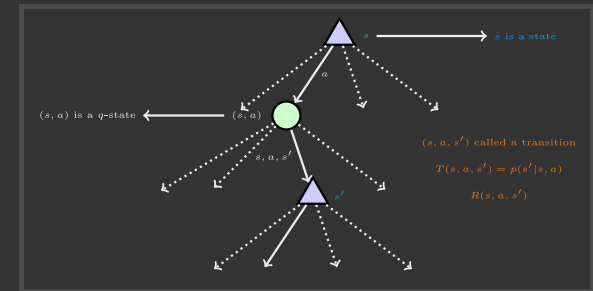
The value (utility) of a state s : $V^*(s) =$ expected utility starting in s and acting optimally



OPTIMAL QUANTITIES

The value (utility) of a state s : $V^*(s) =$ expected utility starting in s and acting optimally

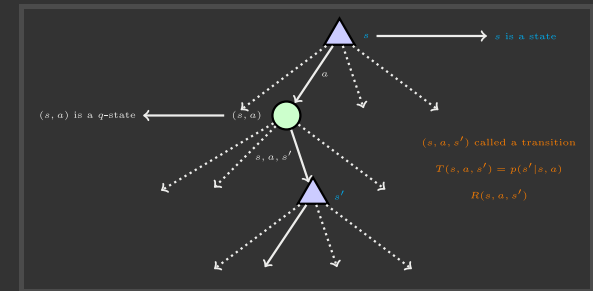
The value (utility) of a q -state (s, a) : $Q^*(s, a) =$ expected utility starting out having taken action a from state s and (thereafter) acting optimally



OPTIMAL QUANTITIES

The value (utility) of a state s : $V^*(s) =$ expected utility starting in s and acting optimally

The value (utility) of a q -state (s, a) : $Q^*(s, a) =$ expected utility starting out having taken action a from state s and (thereafter) acting optimally

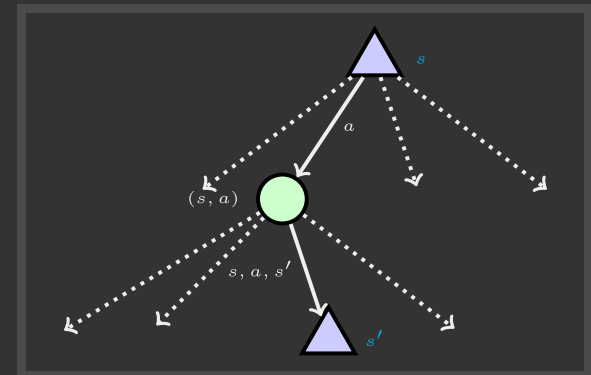


The optimal policy: $\pi^*(s) =$ optimal action from state s

VALUES OF STATES

Fundamental operation: compute the (expectimax) value of a state

- Expected utility under optimal action
- Average sum of (discounted) rewards
- This is just what expectimax computed

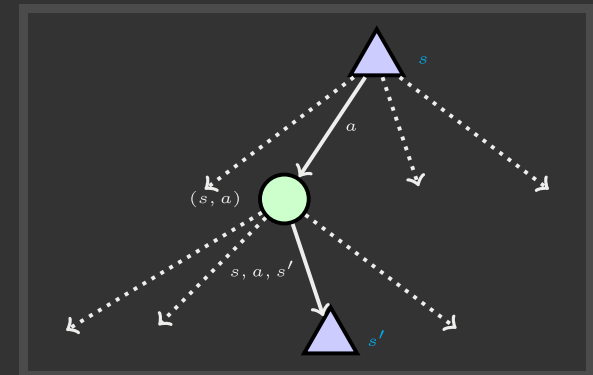


VALUES OF STATES

Fundamental operation: compute the (expectimax) value of a state

- Expected utility under optimal action
- Average sum of (discounted) rewards
- This is just what expectimax computed

Recursive definition of value:



VALUES OF STATES

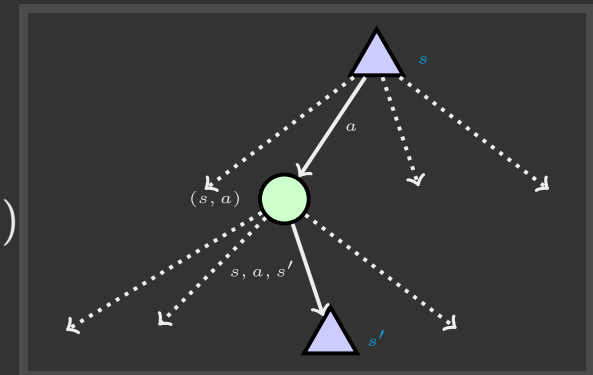
Fundamental operation: compute the (expectimax) value of a state

- Expected utility under optimal action
- Average sum of (discounted) rewards
- This is just what expectimax computed

Recursive definition of value:

$$V^*(s) = \max_a Q^*(s, a)$$

(5)



VALUES OF STATES

Fundamental operation: compute the (expectimax) value of a state

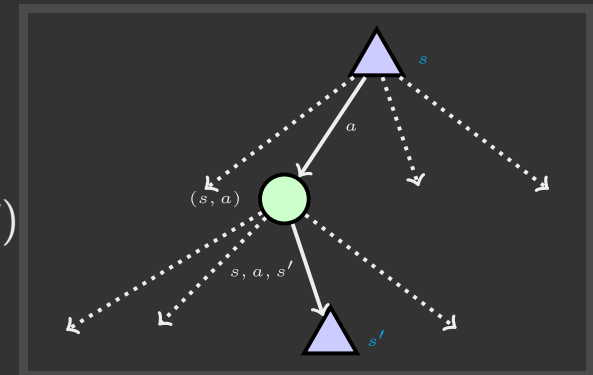
- Expected utility under optimal action
- Average sum of (discounted) rewards
- This is just what expectimax computed

Recursive definition of value:

$$V^*(s) = \max_a Q^*(s, a)$$

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

(7)



VALUES OF STATES

Fundamental operation: compute the (expectimax) value of a state

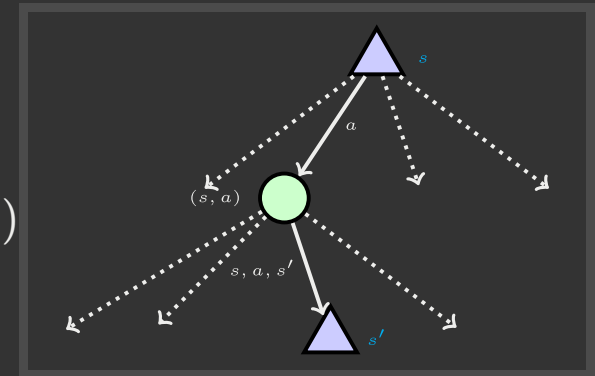
- Expected utility under optimal action
- Average sum of (discounted) rewards
- This is just what expectimax computed

Recursive definition of value:

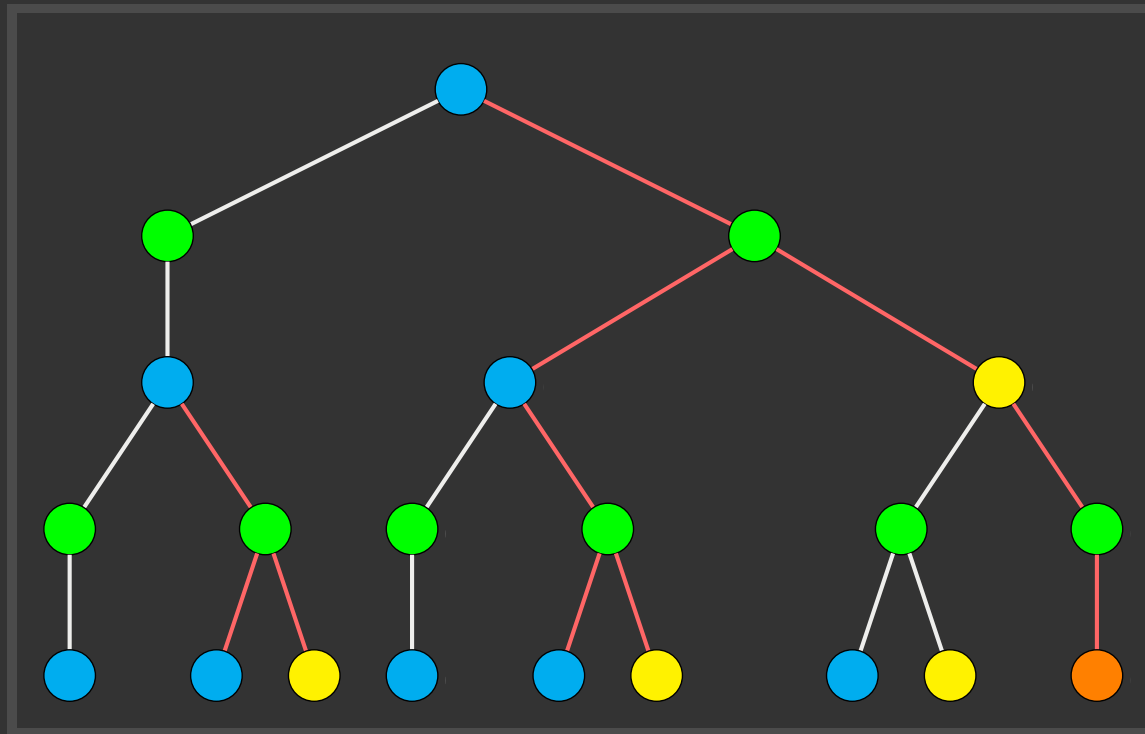
$$V^*(s) = \max_a Q^*(s, a)$$

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')] \quad (10)$$



RACING SEARCH TREE



RACING SEARCH TREE

We are doing way too much work with expectimax.



RACING SEARCH TREE

We are doing way too much work with expectimax.

Problem 1: States are repeated



RACING SEARCH TREE

We are doing way too much work with expectimax.

Problem 1: States are repeated

- Idea: Only compute needed quantities once



RACING SEARCH TREE

We are doing way too much work with expectimax.

Problem 1: States are repeated

- Idea: Only compute needed quantities once

Problem 2: Tree goes on forever



RACING SEARCH TREE

We are doing way too much work with expectimax.

Problem 1: States are repeated

- Idea: Only compute needed quantities once

Problem 2: Tree goes on forever

- Idea: Do a depth-limited computation, but with increasing depths until change is small



RACING SEARCH TREE

We are doing way too much work with expectimax.

Problem 1: States are repeated

- Idea: Only compute needed quantities once

Problem 2: Tree goes on forever

- Idea: Do a depth-limited computation, but with increasing depths until change is small
- Note: deep parts of the tree eventually do not matter if $\gamma < 1$



TIME-LIMITED VALUES

Key idea: time-limited values



TIME-LIMITED VALUES

Key idea: time-limited values

Define $V_k(s)$ to be the optimal value of s if the game ends in k more time steps



TIME-LIMITED VALUES

Key idea: time-limited values

Define $V_k(s)$ to be the optimal value of s if the game ends in k more time steps

- Equivalently, it is what a depth- k expectimax would give from s

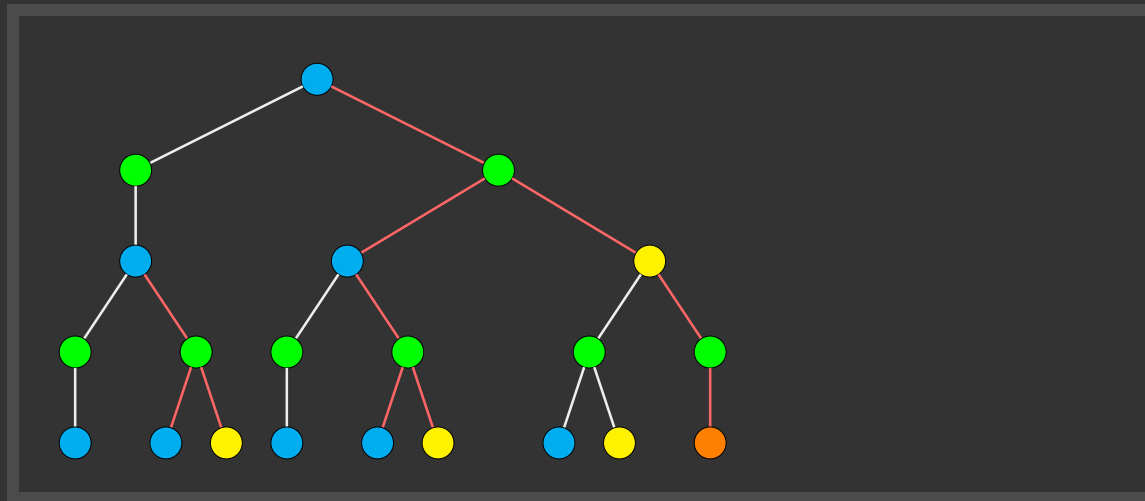


TIME-LIMITED VALUES

Key idea: time-limited values

Define $V_k(s)$ to be the optimal value of s if the game ends in k more time steps

- Equivalently, it is what a depth- k expectimax would give from s

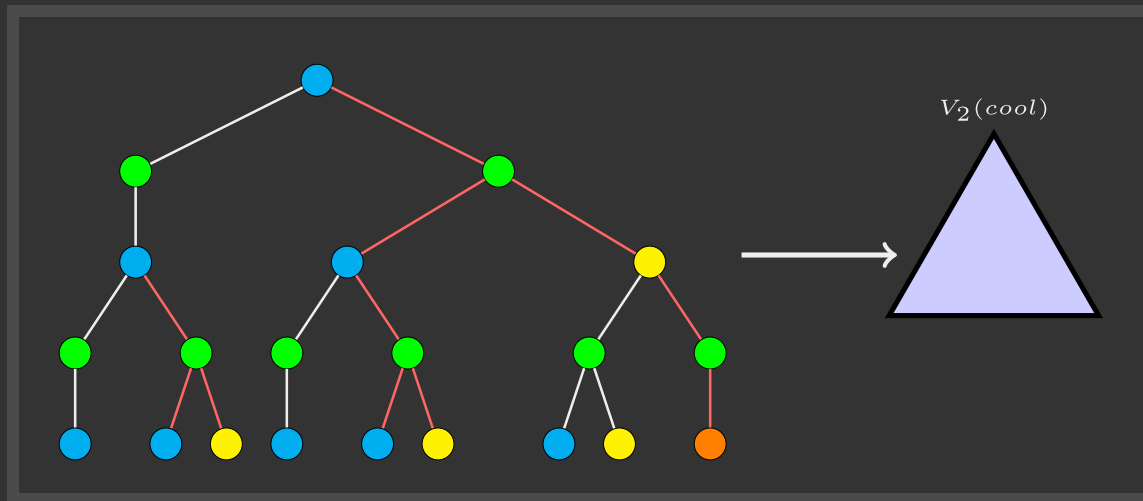


TIME-LIMITED VALUES

Key idea: time-limited values

Define $V_k(\mathbf{s})$ to be the optimal value of $\mathbf{s}$ if the game ends in k more time steps

- Equivalently, it is what a depth- k expectimax would give from $\mathbf{s}$

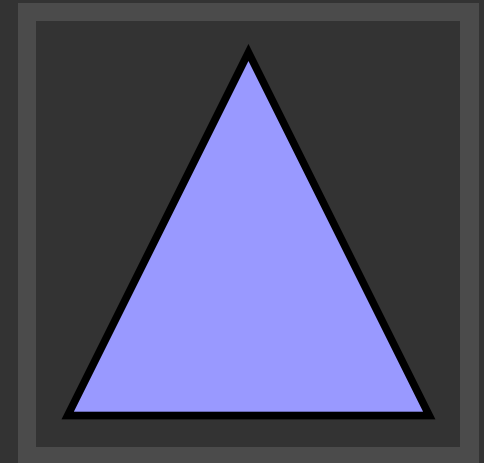
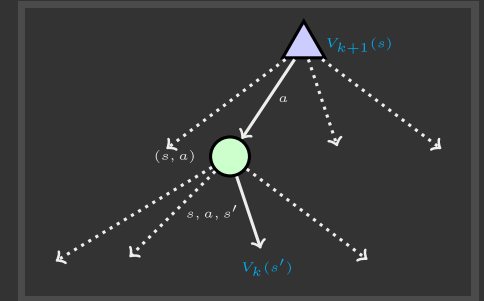


VALUE ITERATION



VALUE ITERATION

Start with $V_0(s) = 0$: no time steps left means an expected reward sum of zero

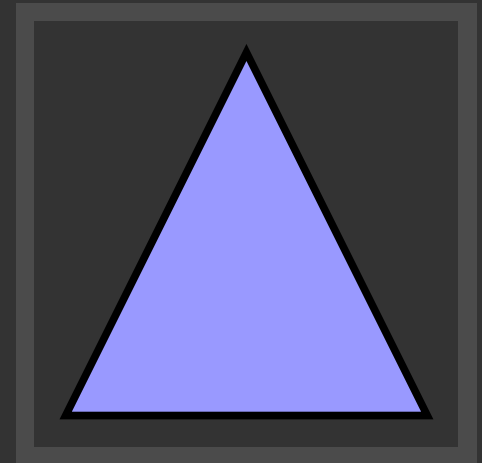
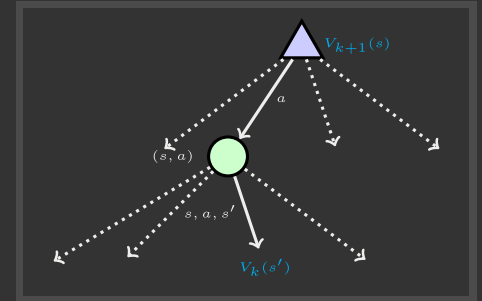


VALUE ITERATION

Start with $V_0(s) = 0$: no time steps left means an expected reward sum of zero

Given vector of $V_k(s)$ values, do one ply of expectimax from each state:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$



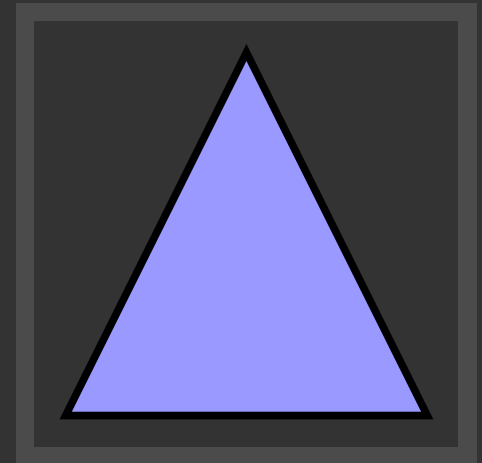
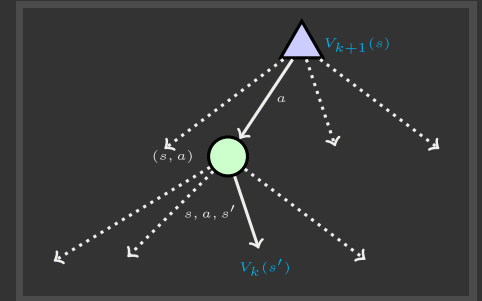
VALUE ITERATION

Start with $V_0(s) = 0$: no time steps left means an expected reward sum of zero

Given vector of $V_k(s)$ values, do one ply of expectimax from each state:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

Repeat until convergence



VALUE ITERATION

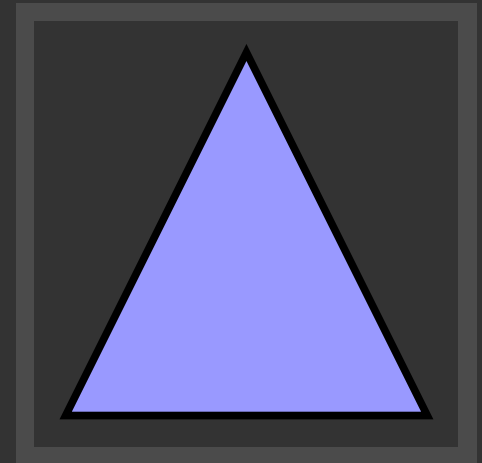
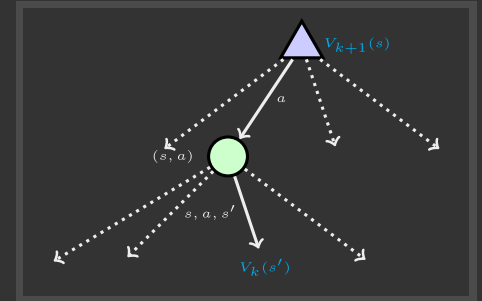
Start with $V_0(s) = 0$: no time steps left means an expected reward sum of zero

Given vector of $V_k(s)$ values, do one ply of expectimax from each state:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

Repeat until convergence

Complexity of each iteration: $\mathcal{O}(S^2 A)$



VALUE ITERATION

Start with $V_0(s) = 0$: no time steps left means an expected reward sum of zero

Given vector of $V_k(s)$ values, do one ply of expectimax from each state:

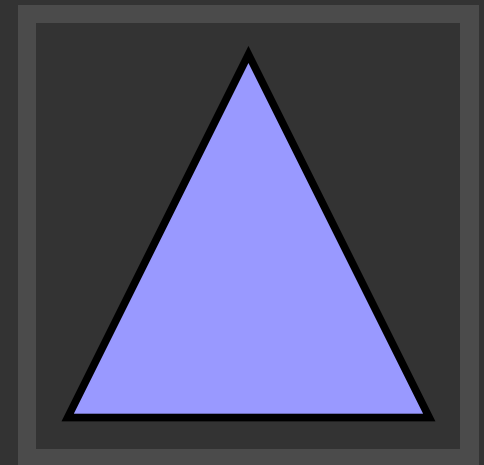
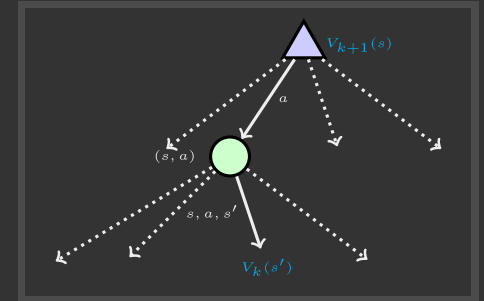
$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

Repeat until convergence

Complexity of each iteration: $\mathcal{O}(S^2 A)$

Theorem: will converge to unique optimal values

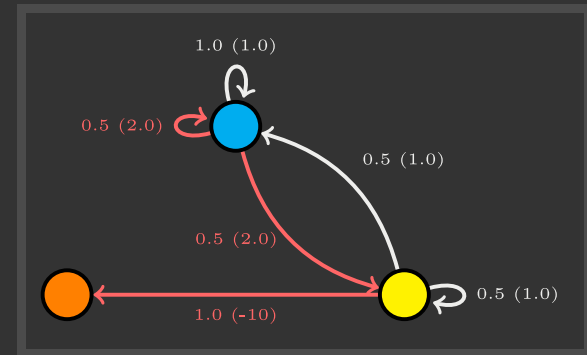
- Basic idea: approximations get refined towards optimal values
- Policy may converge long before values do



EXAMPLE VALUE ITERATION

Cool **Warm** **Overheated**

V_2			
V_1			
V_0	0	0	0



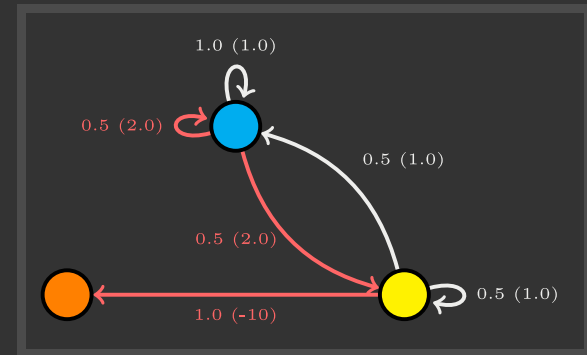
$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + V_k(s')]$$

Assume no discount



EXAMPLE VALUE ITERATION

	Cool	Warm	Overheated
V_2	3.5	2.5	0
V_1	2	1	0
V_0	0	0	0

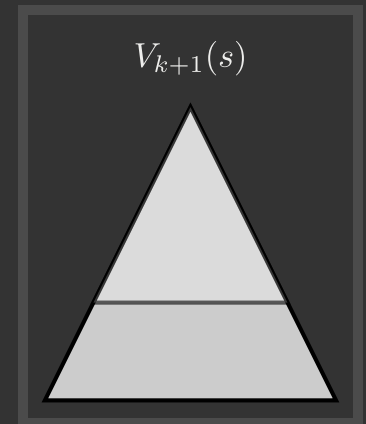
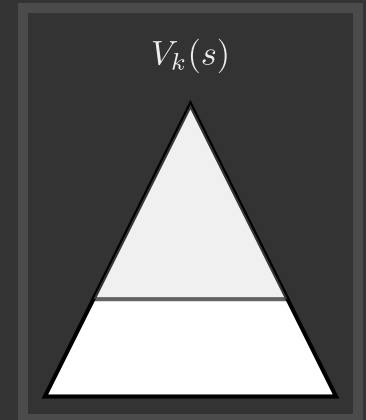


$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + V_k(s')]$$

Assume no discount

CONVERGENCE

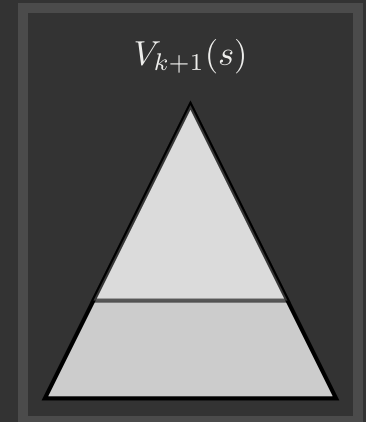
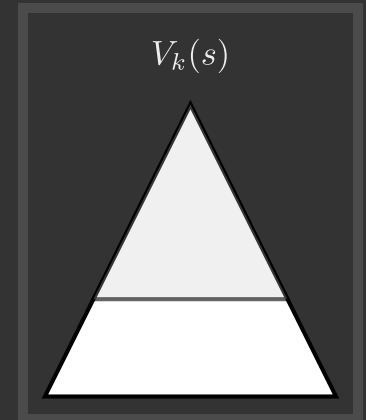
How do we know the V_k vectors are going to converge?



CONVERGENCE

How do we know the V_k vectors are going to converge?

Case 1: If the tree has maximum depth M , then V_M holds the actual untruncated values



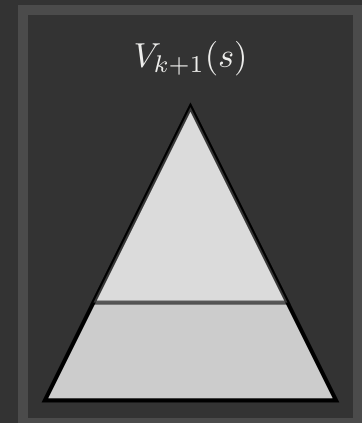
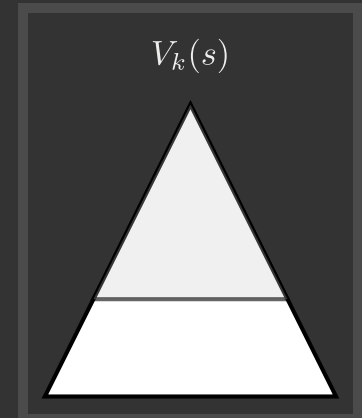
CONVERGENCE

How do we know the V_k vectors are going to converge?

Case 1: If the tree has maximum depth M , then V_M holds the actual untruncated values

Case 2: If the discount is less than 1

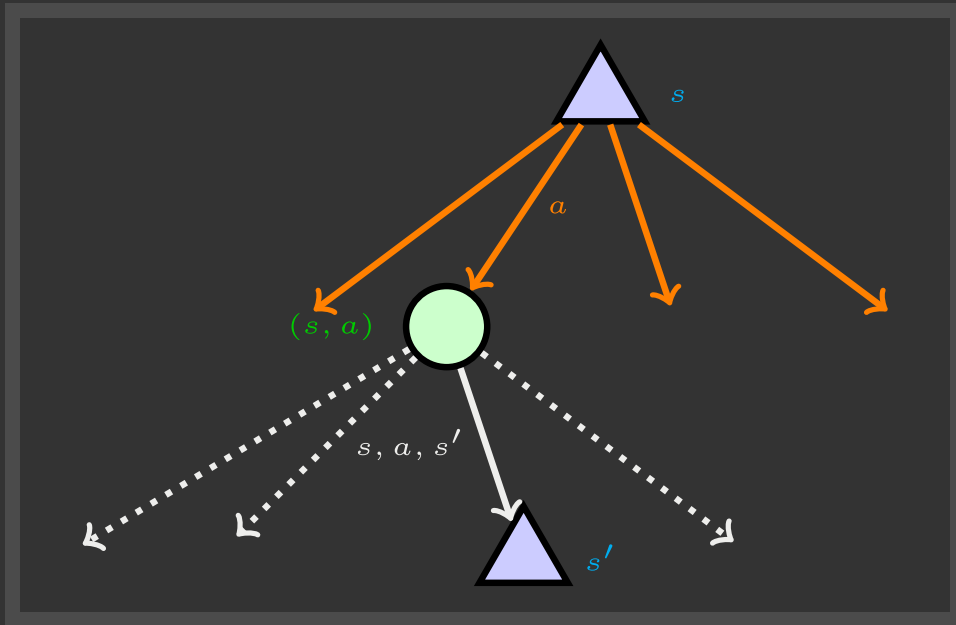
- Sketch: For any state V_k and V_{k+1} can be viewed as depth $k + 1$ expectimax results in nearly identical search trees
- The difference is that on the bottom layer, V_{k+1} has actual rewards while V_k has zeros
- That last layer is at best all R_{MAX}
- It is at worst R_{MIN}
- But everything is discounted by γ^k that far out
- So V_k and V_{k+1} are at most $\gamma^k \max |R|$ different
- So as k increases, the values converge



POLICY EVALUATION

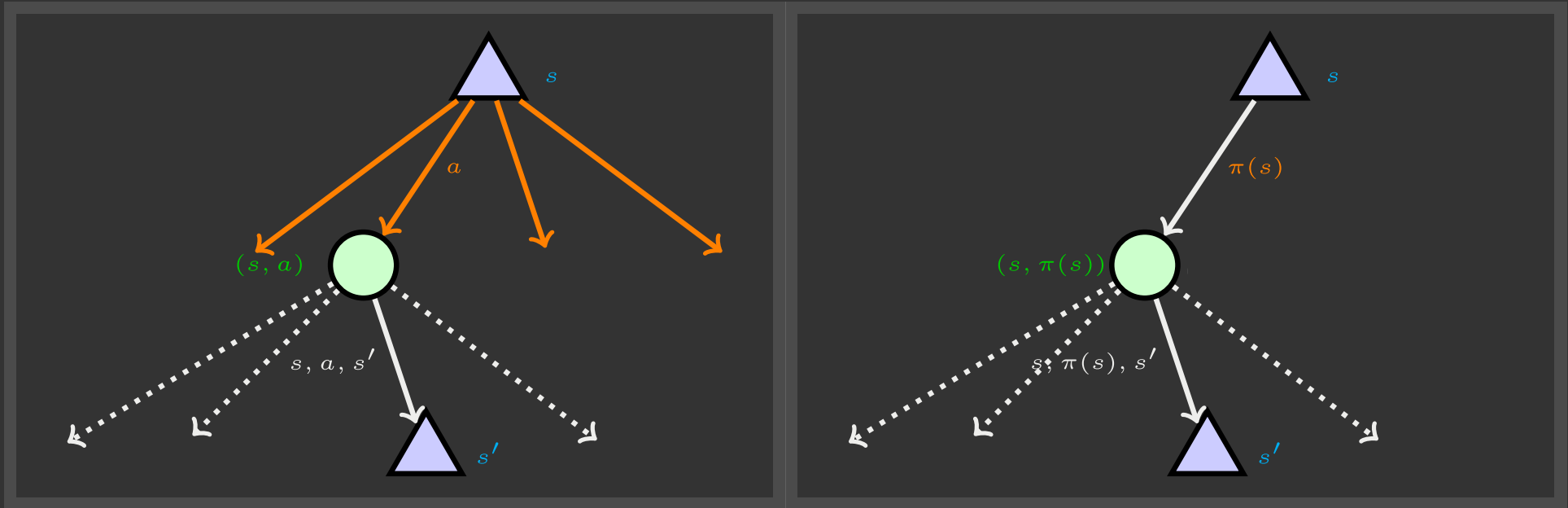


FIXED POLICIES



Expectimax trees max over all actions to compute the optimal values

FIXED POLICIES



Expectimax trees max over all actions to compute the optimal values

If we fixed some policy $\pi(s)$, then the tree would be simpler – only one action per state

- ... though the tree's value would depend on which policy we fixed

UTILITIES FOR A FIXED POLICY

Another basic operation: compute the utility of a state s under a fixed (generally non-optimal) policy

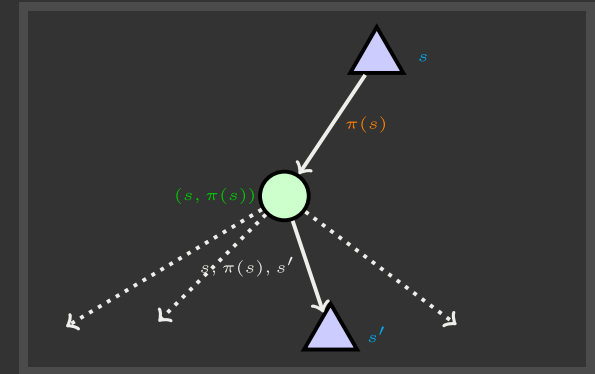


UTILITIES FOR A FIXED POLICY

Another basic operation: compute the utility of a state s under a fixed (generally non-optimal) policy

Define the utility of a state s , under a fixed policy π :

$V^\pi(s)$ = expected total discounted rewards starting in s and following π



UTILITIES FOR A FIXED POLICY

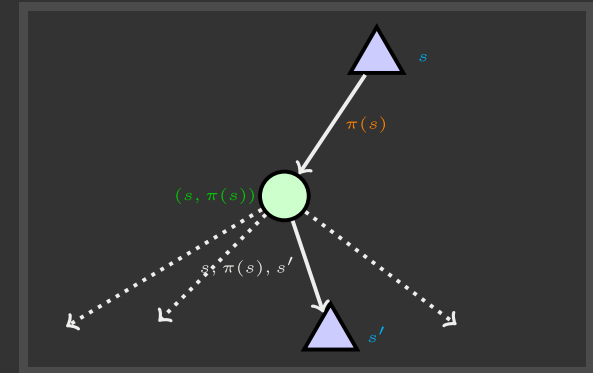
Another basic operation: compute the utility of a state s under a fixed (generally non-optimal) policy

Define the utility of a state s , under a fixed policy π :

$V^\pi(s)$ = expected total discounted rewards starting in s and following π

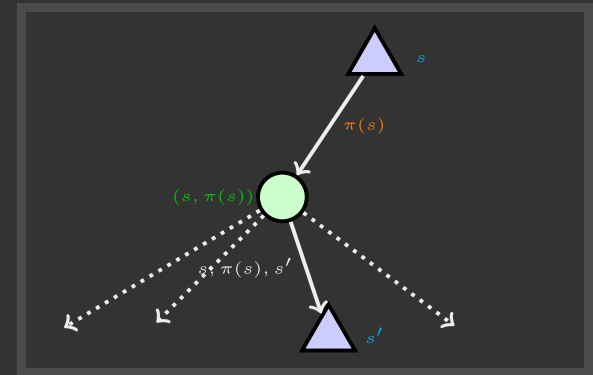
Recursive relation (one-step look-ahead / Bellman equation):

$$V^\pi(s) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^\pi(s')]$$



POLICY EVALUATION

How do we calculate the V 's for a fixed policy π ?

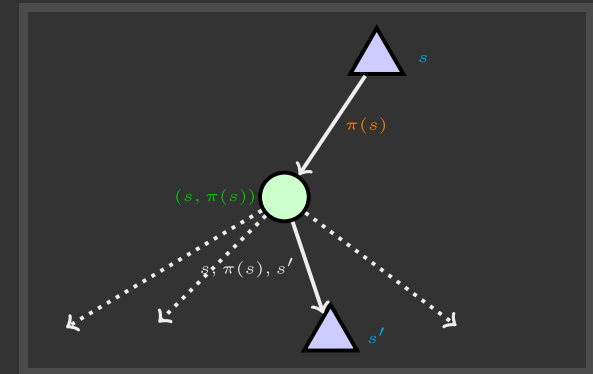


POLICY EVALUATION

How do we calculate the V 's for a fixed policy π ?

Idea 1: Turn recursive Bellman equations into updates (like value iteration)

$$V_0^\pi(s) = 0$$
$$V_{k+1}^\pi(s) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k^\pi(s')]$$



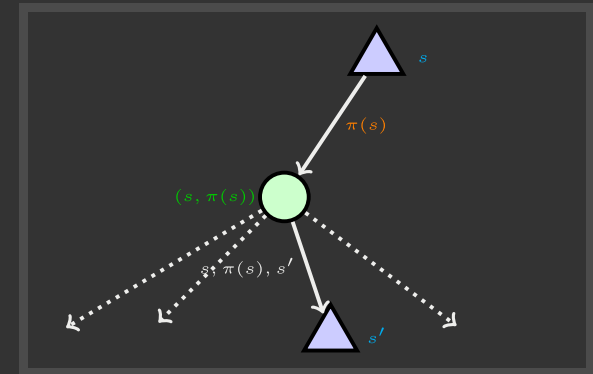
POLICY EVALUATION

How do we calculate the V 's for a fixed policy π ?

Idea 1: Turn recursive Bellman equations into updates (like value iteration)

$$V_0^\pi(s) = 0$$
$$V_{k+1}^\pi(s) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k^\pi(s')]$$

Efficiency: $\mathcal{O}(S^2)$ per iteration



POLICY EVALUATION

How do we calculate the V 's for a fixed policy π ?

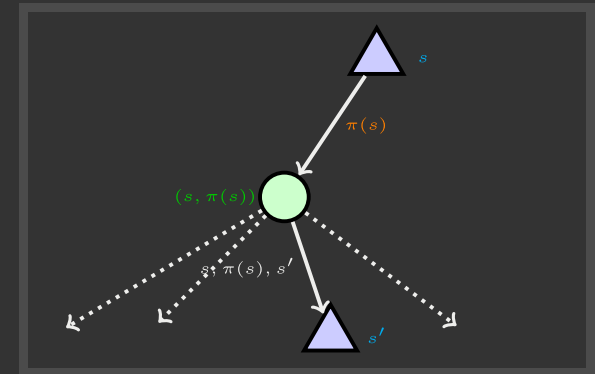
Idea 1: Turn recursive Bellman equations into updates (like value iteration)

$$V_0^\pi(s) = 0$$
$$V_{k+1}^\pi(s) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k^\pi(s')]$$

Efficiency: $\mathcal{O}(S^2)$ per iteration

Idea 2: Without the maxes, the Bellman equations are just a linear system

- Solve with Matlab (or your favorite linear system solver)



POLICY EXTRACTION



COMPUTING VALUES FROM ACTIONS

Imagine we have the optimal values $V^*(s)$



COMPUTING VALUES FROM ACTIONS

Imagine we have the optimal values $V^*(s)$

How should we act?



COMPUTING VALUES FROM ACTIONS

Imagine we have the optimal values $V^*(s)$

How should we act?

- It is not obvious



COMPUTING VALUES FROM ACTIONS

Imagine we have the optimal values $V^*(s)$

How should we act?

- It is not obvious

We need to do a mini-expectimax (one step)

$$\pi^*(s) = \operatorname{argmax}_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$



COMPUTING VALUES FROM ACTIONS

Imagine we have the optimal values $V^*(s)$

How should we act?

- It is not obvious

We need to do a mini-expectimax (one step)

$$\pi^*(s) = \operatorname{argmax}_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

This is called [policy extraction](#), since it gets the policy implied by the values.



COMPUTING ACTIONS FROM Q-VALUES

Imagine we have the optimal q-values: $Q^*(s, a)$



COMPUTING ACTIONS FROM Q-VALUES

Imagine we have the optimal q-values: $Q^*(s, a)$

How should we act?



COMPUTING ACTIONS FROM Q-VALUES

Imagine we have the optimal q-values: $Q^*(s, a)$

How should we act?

- Completely trivial to decide !!

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$$



COMPUTING ACTIONS FROM Q-VALUES

Imagine we have the optimal q-values: $Q^*(s, a)$

How should we act?

- Completely trivial to decide !!

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$$

Important lesson: actions are easier to select from q-values than values !!



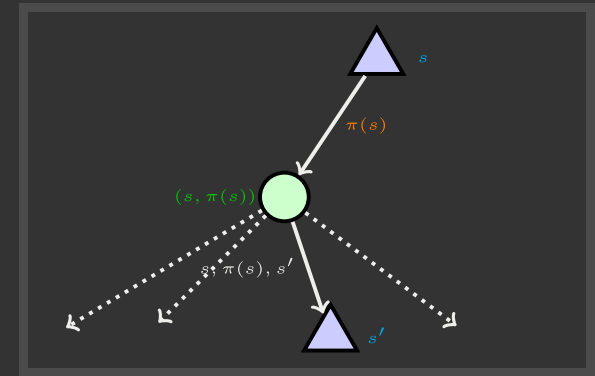
POLICY ITERATION



PROBLEMS WITH VALUE ITERATION

Value iteration repeats the Bellman updates:

$$V_{k+1}^* = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k^*(s')]$$



PROBLEMS WITH VALUE ITERATION

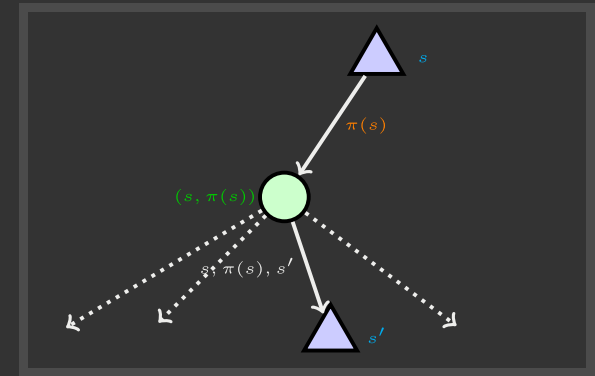
Value iteration repeats the Bellman updates:

$$V_{k+1}^* = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k^*(s')]$$

Problem 1: It is slow – $\mathcal{O}(S^2 A)$ per iteration

Problem 2: The "max" at each state rarely changes

Problem 3: The policy often converges long before the values



POLICY ITERATION

Alternative approach for optimal values:



POLICY ITERATION

Alternative approach for optimal values:

- **Step 1: Policy evaluation:** calculate utilities for some fixed policy (not optimal utilities) until convergence
- **Step 2: Policy improvement:** update policy using one-step look-ahead with resulting converged (but not optimal) utilities as future values
- Repeat steps until policy converges



POLICY ITERATION

Alternative approach for optimal values:

- **Step 1: Policy evaluation:** calculate utilities for some fixed policy (not optimal utilities) until convergence
- **Step 2: Policy improvement:** update policy using one-step look-ahead with resulting converged (but not optimal) utilities as future values
- Repeat steps until policy converges

This is policy iteration

- It is still optimal !!
- Can converge (much) faster under some conditions



POLICY ITERATION

Evaluation: For fixed current policy π , find values with policy evaluation:



POLICY ITERATION

Evaluation: For fixed current policy π , find values with policy evaluation:

- Iterate until values converge:

$$V_{k+1}^{\pi_i}(s) = \sum_{s'} T(s, \pi_i(s), s') [R(s, \pi_i(s), s') + \gamma V_k^{\pi_i}(s')]$$



POLICY ITERATION

Evaluation: For fixed current policy π , find values with policy evaluation:

- Iterate until values converge:

$$V_{k+1}^{\pi_i}(s) = \sum_{s'} T(s, \pi_i(s), s') [R(s, \pi_i(s), s') + \gamma V_k^{\pi_i}(s')]$$

Improvement: For fixed values, get a better policy using policy extraction



POLICY ITERATION

Evaluation: For fixed current policy π , find values with policy evaluation:

- Iterate until values converge:

$$V_{k+1}^{\pi_i}(s) = \sum_{s'} T(s, \pi_i(s), s') [R(s, \pi_i(s), s') + \gamma V_k^{\pi_i}(s')]$$

Improvement: For fixed values, get a better policy using policy extraction

- One-step look-ahead:

$$\pi_{i+1}(s) = \operatorname{argmax}_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^{\pi_i}(s')]$$



COMPARISON

Both value iteration and policy iteration compute the same thing (all optimal values)



COMPARISON

Both value iteration and policy iteration compute the same thing (all optimal values)

In value iteration:

- Every iteration updates both the values and (implicitly) the policy
- We do not track the policy, but taking the max over actions implicitly recomputes it.



COMPARISON

Both value iteration and policy iteration compute the same thing (all optimal values)

In value iteration:

- Every iteration updates both the values and (implicitly) the policy
- We do not track the policy, but taking the max over actions implicitly recomputes it.

In policy iteration:

- We do several passes that update utilities with fixed policy (each pass is fast because we consider only one action, not all of them)
- After the policy is evaluated, a new policy is chosen (slow like a value iteration pass)
- The new policy will be better (or we are done)



COMPARISON

Both value iteration and policy iteration compute the same thing (all optimal values)

In value iteration:

- Every iteration updates both the values and (implicitly) the policy
- We do not track the policy, but taking the max over actions implicitly recomputes it.

In policy iteration:

- We do several passes that update utilities with fixed policy (each pass is fast because we consider only one action, not all of them)
- After the policy is evaluated, a new policy is chosen (slow like a value iteration pass)
- The new policy will be better (or we are done)

Both are dynamic programs for solving MDPs



SUMMARY: MDP ALGORITHMS

So you want to . . .

- Compute optimal values: use value iteration or policy iteration
- Compute values for a particular policy: use policy evaluation
- Turn your values into a policy: use policy extraction (one-step lookahead)



SUMMARY: MDP ALGORITHMS

So you want to . . .

- Compute optimal values: use value iteration or policy iteration
- Compute values for a particular policy: use policy evaluation
- Turn your values into a policy: use policy extraction (one-step lookahead)

These all look the same . . .

- They basically are – they are all variations of Bellman updates
- They all use one-step lookahead expectimax fragments
- They differ only in whether we plug in a fixed policy or max over actions





